

Full-Stack Interview Questions & Answers

1. What is the difference between `<div>` and `<span>`?

Answer: `<div>` is block-level and used for layout or structure, while `<span>` is inline and used for styling parts of text.

How to Deliver: "In technical terms, `<div>` is for structural sections, and `<span>` is for inline styling."

2. What are semantic HTML elements? Why are they important?

Answer: Semantic elements clearly describe their meaning in HTML, like `<header>`, `<article>`, `<footer>`. They improve accessibility, SEO, and code readability.

How to Deliver: "Semantic HTML improves structure, SEO, and accessibility by using meaningful tags."

3. What is the difference between `<div>` and `<section>`?

Answer: `<div>` is a generic container, while `<section>` represents a standalone section of content with semantic meaning.

How to Deliver: "<section> is semantic, <div> is generic — use <section> for meaningful sections of a page."

4. What is the difference between inline, block, and inline-block elements in CSS?

Answer: Inline elements don't start on a new line and only take up as much width as needed. Block elements start on a new line and take full width. Inline-block allows elements to flow inline but respect width and height.

How to Deliver: "Inline fits within text, block takes full width, and inline-block mixes both behaviors."

5. What are CSS pseudo-classes and pseudo-elements?

Answer: Pseudo-classes define a special state of an element (e.g., `:hover`, `:focus`). Pseudo-elements style specific parts of elements (e.g., `::before`, `::after`).

How to Deliver: "Pseudo-classes target states, pseudo-elements target parts of elements."

6. What is the difference between relative, absolute, fixed, and sticky positioning in CSS?

Answer:

-

Relative: positioned relative to itself.

- Absolute: positioned relative to nearest positioned ancestor.
- Fixed: relative to viewport, doesn't move on scroll.
- Sticky: switches between relative and fixed based on scroll.
How to Deliver: "Relative is self-based, absolute is parent-based, fixed is viewport-based, and sticky is hybrid."

7. What are media queries in CSS?

Answer: Media queries apply styles based on device characteristics like width, height, or orientation.

How to Deliver: "Media queries make responsive designs by applying styles based on screen size."

8. What is the difference between em, rem, %, px in CSS units?

Answer:

- px: fixed size.
- %: relative to parent.
- em: relative to parent font size.
- rem: relative to root font size.
How to Deliver: "Use rem for consistent scaling, em for relative, % for parent-based, px for fixed."

9. What is the difference between inline CSS, internal CSS, and external CSS?

Answer: Inline CSS applies directly on elements, internal CSS is in <style> tag within a page, and external CSS is in separate .css files.

How to Deliver: "Inline is one element, internal is within the page, external is reusable across pages."

10. What are CSS Flexbox and CSS Grid?

Answer: Flexbox arranges items in one dimension (row/column). Grid arranges items in two dimensions (rows and columns).

How to Deliver: "Flexbox is one-dimensional, Grid is two-dimensional layout system."

11. What is the difference between relative and absolute URLs?

Answer: Relative URLs depend on the current page's path, while absolute URLs specify the full path including domain.

How to Deliver: "Relative changes with page, absolute always includes full domain."

12. What are cookies, localStorage, and sessionStorage in JavaScript?

Answer:

- Cookies: stored in browser, small size, sent with requests.
- localStorage: persistent storage until cleared.
- sessionStorage: lasts until the session/browser closes.
How to Deliver: "Cookies are for server comms, localStorage is permanent, sessionStorage is temporary."

13. What is the difference between == and === in JavaScript?

Answer: == compares values with type coercion, === compares values without type coercion (strict equality).

How to Deliver: "==" checks values loosely, === checks values strictly."

14. What are closures in JavaScript?

Answer: A closure is a function that retains access to its outer scope variables even after the outer function has executed.

How to Deliver: "Closures allow functions to remember variables outside their scope."

15. What is the difference between var, let, and const in JavaScript?

Answer:

- var: function-scoped, hoisted.
- let: block-scoped, not hoisted in the same way.
- const: block-scoped, cannot be reassigned.
How to Deliver: "Use let for mutable, const for constants, avoid var."

16. What is event delegation in JavaScript?

Answer: Event delegation is attaching an event listener to a parent element to handle events on its child elements using event bubbling.

How to Deliver: "Event delegation handles multiple child events by attaching a listener to the parent."

17. What is the difference between synchronous and asynchronous programming?

Answer: Synchronous code runs line by line, blocking execution. Asynchronous code allows non-blocking operations using callbacks, promises, or async/await.

How to Deliver: "Synchronous blocks, asynchronous doesn't block and uses promises/async."

18. What are promises in JavaScript?

Answer: Promises represent a value that may be available now, later, or never. States: pending, resolved, rejected.

How to Deliver: "Promises handle async operations with resolve or reject states."

19. What is the difference between null and undefined?

Answer: undefined means a variable has been declared but not assigned. null is an intentional empty value.

How to Deliver: "Undefined is absence of assignment, null is intentional emptiness."

20. What is hoisting in JavaScript?

Answer: Hoisting moves variable and function declarations to the top of their scope before code execution.

How to Deliver: "Hoisting makes declarations available before actual code execution."

21. What are higher-order functions in JavaScript?

Answer: Functions that take other functions as arguments or return functions.

How to Deliver: "Higher-order functions work with other functions as inputs or outputs."

22. What are arrow functions in JavaScript?

Answer: Concise function syntax with lexical this binding.

How to Deliver: "Arrow functions are shorter and keep the parent's this."

23. What is the difference between map, forEach, filter, and reduce?

Answer:

- map: returns new array with transformed items.

- forEach: iterates but returns nothing.

- filter: returns array of matching items.

- reduce: reduces array to single value.

How to Deliver: "Map transforms, forEach iterates, filter selects, reduce accumulates."

24. What are RESTful APIs?

Answer: REST APIs use HTTP methods (GET, POST, PUT, DELETE) for CRUD operations and follow stateless, resource-based principles.

How to Deliver: "REST API uses HTTP methods to perform CRUD on resources."

25. What is the difference between GET and POST methods?

Answer: GET requests data with URL params, visible in the address bar, limited length. POST sends data in body, more secure, no length restriction.

How to Deliver: "GET retrieves data in URL, POST sends data in body."

26. What are HTTP status codes?

Answer:

- 2xx: Success (200 OK)
 - 3xx: Redirection (301 Moved Permanently)
 - 4xx: Client error (404 Not Found)
 - 5xx: Server error (500 Internal Server Error)
- How to Deliver: "2xx is success, 3xx redirection, 4xx client error, 5xx server error."

27. What is CORS in web development?

Answer: Cross-Origin Resource Sharing controls resource requests from different origins using headers.

How to Deliver: "CORS allows or blocks resource access between different origins."

28. What is JSON and why is it used?

Answer: JSON (JavaScript Object Notation) is a lightweight format for storing and transporting data, human-readable and easy to parse.

How to Deliver: "JSON is a lightweight format for exchanging structured data."

29. What is the difference between SQL and NoSQL databases?

Answer: SQL uses structured schema and relations (tables). NoSQL is schema-less, stores data as documents, key-value, or graphs.

How to Deliver: "SQL is relational and structured, NoSQL is flexible and schema-less."

30. What are primary keys and foreign keys in SQL?

Answer: Primary key uniquely identifies a record. Foreign key establishes relation between two tables.

How to Deliver: "Primary is unique identifier, foreign key links tables."

31. What is normalization in databases?

Answer: Normalization organizes data to reduce redundancy and dependency. Types: 1NF, 2NF, 3NF.

How to Deliver: "Normalization removes redundancy and ensures consistency."

32. What is indexing in databases?

Answer: Indexes improve query speed by creating quick access paths to rows, like a book index.

How to Deliver: "Indexing speeds up queries but may slow inserts/updates."

33. What are ACID properties in databases?

Answer: Atomicity, Consistency, Isolation, Durability ensure reliable transactions.

How to Deliver: "ACID ensures transactions are reliable and consistent."

34. What are database transactions?

Answer: A transaction is a sequence of operations performed as a single logical unit of work, following ACID principles.

How to Deliver: "Transaction is a group of operations executed together reliably."

35. What is the difference between SQL JOIN types?

Answer:

-

INNER JOIN: common records.

-

LEFT JOIN: all from left, matching from right.

-

RIGHT JOIN: all from right, matching from left.

-

FULL JOIN: all records from both sides.

How to Deliver: "Inner is common, left/right keeps one side, full keeps all."

36. What is an ORM (Object Relational Mapping)?

Answer: ORM maps database tables to objects, allowing database operations using object-oriented code (e.g., Hibernate, SQLAlchemy).

How to Deliver: "ORM lets you work with DB tables as objects in code."

37. What is server-side rendering (SSR) vs client-side rendering (CSR)?

Answer: SSR renders HTML on the server before sending to client, faster initial load. CSR renders content in browser using JS, slower initial load but faster navigation.

How to Deliver: "SSR is server-rendered HTML, CSR renders in browser using JS."

38. What is the difference between monolithic and microservices architecture?

Answer: Monolithic apps are single units with tightly coupled components. Microservices are independent, modular services communicating via APIs.

How to Deliver: "Monolithic is one big unit, microservices are independent small services."

39. What are WebSockets?

Answer: WebSockets provide full-duplex communication between client and server over a single persistent connection.

How to Deliver: "WebSockets allow real-time, two-way communication between client and server."

40. What is the difference between authentication and authorization?

Answer: Authentication verifies identity (login), authorization determines access rights

(permissions).

How to Deliver: "Authentication is who you are, authorization is what you can access."

41. What is JWT (JSON Web Token)?

Answer: JWT is a compact, URL-safe token used for authentication and information exchange. It has three parts: header, payload, signature.

How to Deliver: "JWT securely transmits claims between client and server, often for stateless authentication."

42. What is the difference between session-based and token-based authentication?

Answer:

- Session-based: server stores session info, client sends session ID.
- Token-based: client stores token (e.g., JWT), server validates token without storing session.
How to Deliver: "Sessions are server-side, tokens are client-side and stateless."

43. What are REST API best practices?

Answer:

- Use proper HTTP methods (GET, POST, PUT, DELETE).
- Stateless operations.
- Use proper status codes.
- Version your API.

-

Use consistent naming conventions.

How to Deliver: "Follow HTTP methods, statelessness, proper codes, and versioning for REST APIs."

44. What is the difference between PUT and PATCH?

Answer: PUT replaces the entire resource, PATCH updates part of the resource.

How to Deliver: "PUT is full update, PATCH is partial update."

45. What is GraphQL?

Answer: GraphQL is a query language for APIs that allows clients to request exactly the data they need.

How to Deliver: "GraphQL gives precise data queries instead of fixed REST responses."

46. What is the difference between SQL INNER JOIN and LEFT JOIN?

Answer: INNER JOIN returns only matching rows from both tables. LEFT JOIN returns all rows from the left table and matching rows from the right table.

How to Deliver: "Inner returns matches, left returns all left table rows with matches."

47. What is the difference between synchronous and asynchronous JavaScript?

Answer: Synchronous runs line by line, blocking next execution. Asynchronous allows code to run in the background without blocking.

How to Deliver: "Sync blocks, async doesn't block and handles delayed operations."

48. What is the difference between call(), apply(), and bind() in JavaScript?

Answer:

-

call(): invokes function with arguments separately.

-

apply(): invokes function with arguments as array.

-

bind(): returns a new function with this bound.

How to Deliver: "Call/apply invoke immediately, bind returns a bound function."

49. What is the difference between localStorage and sessionStorage?

Answer: LocalStorage persists until cleared, SessionStorage lasts only until tab/browser is closed.

How to Deliver: "LocalStorage is persistent, sessionStorage is temporary."

50. What is the difference between var, let, and const?

Answer:

-

var: function-scoped, can be redeclared.

-

let: block-scoped, cannot be redeclared in same scope.

-

const: block-scoped, cannot be reassigned.

How to Deliver: "Var is old-style, let is modern mutable, const is constant."

51. What is the difference between == and ===?

Answer: == checks value only (type coercion), === checks value and type.

How to Deliver: "Use === for strict comparison, == for loose comparison."

52. What are async/await in JavaScript?

Answer: Syntax to handle asynchronous code like synchronous code using Promises.

How to Deliver: "Async/await makes async code easier to read and maintain."

53. What is event bubbling in JavaScript?

Answer: Event triggers from the innermost element up through ancestors.

How to Deliver: "Event starts at target element and bubbles up through parent elements."

54. What is the difference between PUT and POST?

Answer: POST creates a new resource, PUT updates or replaces an existing resource.

How to Deliver: "POST is for creation, PUT is for update/replacement."

55. What is the difference between null and undefined?

Answer: Undefined means variable declared but not assigned; null is assigned as empty.

How to Deliver: "Undefined is accidental absence, null is intentional emptiness."

56. What is the difference between splice() and slice() in JavaScript?

Answer:

- splice(): modifies original array (add/remove).
- slice(): returns a new array without modifying original.
How to Deliver: "Slice is non-destructive, splice is destructive."

57. What is a single-page application (SPA)?

Answer: SPA loads a single HTML page and dynamically updates content via JS without full page reloads.

How to Deliver: "SPA improves performance and user experience by avoiding full reloads."

58. What is the difference between null, undefined, and NaN?

Answer:

- null: intentional empty value.
- undefined: variable declared but not assigned.
- NaN: not a number, usually from invalid numeric operations.

How to Deliver: "Null is empty, undefined is uninitialized, NaN is invalid number."

59. What are WebSockets?

Answer: Persistent, full-duplex communication protocol for real-time data transfer.

How to Deliver: "WebSockets allow client and server to exchange data instantly without reloading."

60. What is the difference between REST and GraphQL?

Answer: REST has fixed endpoints, GraphQL allows clients to specify exactly what data they need.

How to Deliver: "GraphQL is flexible query-based, REST is endpoint-based."

61. What is responsive web design?

Answer: Designing websites to look good on all devices using fluid grids, media queries, and flexible images.

How to Deliver: "Responsive design adapts layout to any screen size."

62. What are semantic HTML elements?

Answer: Elements that give meaning to content, e.g., <header>, <article>, <footer>.

How to Deliver: "Semantic HTML improves accessibility and SEO."

63. What is the difference between display: none and visibility: hidden?

Answer:

-

display: none: removes element from page layout.

-

visibility: hidden: hides element but keeps space reserved.

How to Deliver: "None removes it completely, hidden keeps space."

64. What are CSS transitions and animations?

Answer: Transitions create smooth changes between states, animations can define keyframes for complex effects.

How to Deliver: "Transitions for simple effects, animations for complex sequences."

65. What is the difference between relative, absolute, fixed, and sticky positioning?

Answer:

- Relative: relative to itself.
- Absolute: relative to nearest positioned ancestor.
- Fixed: relative to viewport.
- Sticky: behaves like relative until scrolling threshold.
How to Deliver: "Relative-self, absolute-parent, fixed-viewport, sticky-hybrid."

66. What are CSS pseudo-classes?

Answer: Styles applied to elements based on state, e.g., :hover, :focus.

How to Deliver: "Pseudo-classes target element states dynamically."

67. What are CSS pseudo-elements?

Answer: Target parts of elements, e.g., ::before, ::after.

How to Deliver: "Pseudo-elements style portions of content, not the whole element."

68. What is the difference between inline, inline-block, and block elements?

Answer: Inline flows in text, block takes full width, inline-block flows inline but accepts width/height.

How to Deliver: "Inline-text, block-full, inline-block-hybrid."

69. What is the difference between opacity and visibility in CSS?

Answer: Opacity changes transparency but element still interacts; visibility hides but retains space.

How to Deliver: "Opacity is transparent, visibility hides but keeps space."

70. What are CSS media queries?

Answer: Apply styles based on device characteristics like width or orientation.

How to Deliver: "Media queries make websites responsive."

71. What is the difference between rem and em units in CSS?

Answer:

-

em: relative to parent element's font size.

-

rem: relative to root element's font size.

How to Deliver: "Rem is root-relative, em is parent-relative."

72. What is the difference between CSS Grid and Flexbox?

Answer: Flexbox is one-dimensional (row/column). Grid is two-dimensional (rows and columns).

How to Deliver: "Flexbox for linear layout, Grid for complex 2D layouts."

73. What is z-index in CSS?

Answer: Determines stacking order of overlapping elements. Higher z-index comes in front.

How to Deliver: "Z-index controls which elements appear on top."

74. What are CSS variables?

Answer: Custom properties defined with `--var-name` and accessed using `var(--var-name)`.

How to Deliver: "CSS variables allow reusable and maintainable values."

75. What is the difference between relative and absolute URLs?

Answer: Relative depends on current path, absolute includes full domain.

How to Deliver: "Relative changes with page, absolute always complete."

76. What are cookies, sessionStorage, and localStorage?

Answer:

- Cookies: small, sent with HTTP requests.
- sessionStorage: lasts until browser closes.
- localStorage: persistent until cleared.
How to Deliver: "Cookies for server, sessionStorage for session, localStorage for permanent client storage."

77. What is the difference between SQL and NoSQL?

Answer: SQL is relational with structured schema, NoSQL is schema-less and flexible.

How to Deliver: "SQL is structured, NoSQL is flexible and document-based."

78. What is normalization in SQL?

Answer: Organizing tables to reduce redundancy and dependency.

How to Deliver: "Normalization avoids duplicate data and maintains consistency."

79. What is denormalization in SQL?

Answer: Combining tables to reduce JOINS, trading redundancy for speed.

How to Deliver: "Denormalization improves query speed at the cost of redundancy."

80. What are ACID properties?

Answer: Atomicity, Consistency, Isolation, Durability — ensures reliable transactions.

How to Deliver: "ACID guarantees transactions are safe, consistent, isolated, and durable."

Q81: What is the event loop in JavaScript?

Answer: The event loop handles execution of code, events, and queued tasks, allowing JavaScript (single-threaded) to manage asynchronous operations.

How to Deliver: "I'd explain that the event loop lets JS handle async tasks without blocking the main thread."

Q82: Difference between Promises and async/await?

Answer:

- Promises: Use `.then()` and `.catch()` chaining.
- Async/Await: Cleaner syntax, looks synchronous, still asynchronous under the hood.
How to Deliver: "Async/await is syntactic sugar over Promises, making async code easier to read."

Q83: What is hoisting in JavaScript?

Answer: Variable and function declarations are moved to the top of their scope. Example: `var` is hoisted but initialized as `undefined`.

How to Deliver: "I'd say hoisting means JS reads declarations first, then executes code."

Q84: Difference between shallow copy and deep copy?

Answer:

- Shallow copy: Copies references only (e.g., `Object.assign()`).
- Deep copy: Copies actual values independently (e.g., `structuredClone()` or `JSON.parse(JSON.stringify(obj))`).
How to Deliver: "I'd emphasize shallow copy shares references, while deep copy creates independent duplicates."

Q85: What are JavaScript prototypes?

Answer: Every JS object has a prototype from which it inherits properties and methods. Enables inheritance.

How to Deliver: "I'd say prototypes allow objects to share functionality efficiently."

Q86: Explain debounce and throttle.

Answer:

- Debounce: Delays execution until user stops an action (e.g., search input).
 - Throttle: Executes function at fixed intervals (e.g., scroll events).
How to Deliver: "Debounce reduces unnecessary calls, throttle limits execution rate."
-

Advanced Node.js (Q87–Q91)

Q87: What is middleware in Express.js?

Answer: Functions with access to req, res, and next. Used for logging, authentication, error handling.

How to Deliver: "Middleware is code that processes requests before reaching the final route handler."

Q88: What are streams in Node.js?

Answer: Streams allow reading/writing data piece by piece, avoiding full memory load.

Types: readable, writable, duplex, transform.

How to Deliver: "I'd explain streams are efficient for large files or continuous data."

Q89: How does Node.js handle multiple requests if single-threaded?

Answer: Uses an event loop and asynchronous callbacks. Heavy tasks are delegated to worker threads or libuv.

How to Deliver: "Node.js uses async non-blocking I/O to handle concurrent requests efficiently."

Q90: Difference between process.nextTick() and setImmediate()?

Answer:

- process.nextTick(): Executes immediately after current operation.
- setImmediate(): Executes in next iteration of event loop.
How to Deliver: "I'd highlight the subtle timing difference in execution order."

Q91: How do you secure a Node.js application?

Answer:

- Input validation & sanitization
- Use HTTPS
- Prevent XSS/CSRF
- Use Helmet.js
- Store secrets securely (env vars)
How to Deliver: "I'd emphasize both code-level and server-level security practices."

Database / System Design (Q92–Q97)

Q92: What is indexing in databases?

Answer: Indexes speed up read queries by storing data pointers, but may increase write cost. Example in MongoDB: `db.users.createIndex({email:1})`

How to Deliver: "Indexes are like a book's index — they make searching faster."

Q93: What is database normalization?

Answer: Organizing data to reduce redundancy and improve integrity, e.g., splitting repeated fields into separate tables.

How to Deliver: "Normalization ensures data is stored logically without duplication."

Q94: Difference between SQL transactions and NoSQL atomic operations?

Answer:

- SQL: ACID transactions (atomic, consistent, isolated, durable).
- NoSQL: Atomic at document level (MongoDB), not always multi-document.
How to Deliver: "I'd compare SQL for full ACID compliance, NoSQL for flexibility at document-level."

Q95: What are database joins?

Answer: Combine rows from two or more tables: INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN.

How to Deliver: "I'd explain joins merge data based on related keys."

Q96: What is database sharding?

Answer: Partitioning large databases into smaller pieces across multiple servers for faster access and scalability.

How to Deliver: "Sharding splits data horizontally for high-performance distributed systems."

Q97: How does MongoDB ensure high availability?

Answer: Uses replica sets: one primary, multiple secondary nodes for failover.

How to Deliver: "I'd highlight replication ensures uptime and data safety."

System Design / Full-Stack Scenario Questions (Q98–Q108)

Q98: How would you design a URL shortener (like Bitly)?

Answer:

- Hash function maps long → short URL
- Store mapping in DB (SQL/NoSQL)
- Redirect service expands short → long
- Add caching (Redis)
How to Deliver: "I'd describe hashing, storage, and caching for efficient redirects."

Q99: How do you scale a chat application?

Answer:

- Use WebSockets for real-time messaging
- Store chat history in NoSQL/Redis
- Horizontal scaling with load balancers

- Use message brokers (Kafka/RabbitMQ)
How to Deliver: "Real-time + scalable data storage + load balancing are key."

Q100: Difference between monolith and microservices architecture?

Answer:

- Monolith: Single codebase, tightly coupled
- Microservices: Independent, loosely coupled, communicate via APIs
How to Deliver: "Microservices allow independent scaling and deployment."

Q101: How do you handle caching in full-stack applications?

Answer:

- Browser caching (localStorage, IndexedDB)
- CDN caching for static assets
- Server-side caching (Redis, Memcached)
How to Deliver: "I'd explain caching at multiple layers improves performance."

Q102: What is CAP theorem?

Answer: Distributed systems can only guarantee two of: Consistency, Availability, Partition tolerance.

How to Deliver: "I'd summarize trade-offs when designing distributed apps."

Q103: Design considerations for high-traffic e-commerce platforms (Amazon, Flipkart)?

Answer:

- Scalability (auto-scaling servers, CDN)
 - Security (payments, authentication)
 - Performance (caching, async processing)
 - Fault tolerance (replication, load balancing)
- How to Deliver: "I'd focus on availability, security, and performance."

Q104: If your app suddenly becomes slow, how do you debug?

Answer:

- Check logs for errors
 - Monitor DB queries (indexes, slow queries)
 - Check CPU/memory usage
 - Use profiling tools (DevTools, Node.js profiler)
- How to Deliver: "I'd approach debugging systematically from backend to frontend."

Q105: If an API request takes too long, what strategies would you apply?

Answer:

- Optimize queries
- Add caching
- Use async processing/queues
- Introduce pagination/limit payload
How to Deliver: "I'd optimize at both server and database levels."

Q106: How would you implement file upload in Node.js?

Answer: Use multer middleware in Express to handle multipart/form-data.

How to Deliver: "I'd describe handling uploads securely and efficiently."

Q107: How do you prevent SQL injection or NoSQL injection?

Answer:

- Use parameterized queries (SQL)
- Use ORM (Sequelize, Mongoose)
- Validate and sanitize inputs
How to Deliver: "I'd stress input validation and prepared statements as key security measures."

Q108: How would you implement authentication in a MERN app?

Answer:

- Backend validates user credentials
- Generate JWT token
- Store token in HTTP-only cookie or localStorage
- Protect routes with middleware
How to Deliver: "I'd outline secure token-based authentication for front-end routing."

General Questions

These questions assess your overall experience and approach to development.

- Q1: What are your preferred front-end and back-end technologies, and why?
- A: This is a chance to demonstrate your passion and expertise. Mention the technologies you've used most, like Node.js for the back end and a framework like React or Angular for the front end. Explain your choices based on factors like performance, community support, or personal experience.

-

Q2: Describe a challenging project you've worked on. What were the key challenges and how did you overcome them?

-

A: This question tests your problem-solving skills. Describe a specific project and detail a technical challenge you faced, such as optimizing a slow database query or handling a complex asynchronous operation. Explain your thought process and the steps you took to solve the issue.

-

Q3: How do you ensure the performance and scalability of a web application?

-

A: Discuss both front-end and back-end strategies. For the front end, mention things like code splitting, lazy loading, and responsive design. For the back end, talk about caching, database indexing, and utilizing a microservices architecture.

Frontend Development

These questions relate to the browser-side of an application.

-

Q4: Explain the box model in CSS.

-

A: The box model is a fundamental concept in CSS that describes how elements are rendered on a page. Every element is a rectangular box with four components: the content area, padding, border, and margin.

-

Q5: What is the difference between `==` and `===` in JavaScript?

-

A: The `==` operator checks for loose equality, meaning it compares values after type coercion. The `===` operator checks for strict equality, comparing both the value and the type without any coercion. Use `===` to avoid unexpected behavior.

-

Q6: How does the browser render a webpage?

-

A: The browser first parses HTML to create the Document Object Model (DOM) tree and CSS to create the CSS Object Model (CSSOM). It then combines these into a render tree to determine the layout of each visible element. Finally, it paints the pixels on the screen.

-

Q7: Explain the concept of hoisting in JavaScript.

-

A: Hoisting is a JavaScript behavior where variable and function declarations are moved to the top of their containing scope during the compilation phase, before the code is executed.

-

Q8: What is a closure in JavaScript? Provide an example.

-

A: A closure is a function that remembers its outer scope and can access variables from that scope even after the outer function has finished executing. For example, a function returning another function that uses a variable declared in the outer scope.

-

Q9: What is the purpose of the AJAX?

-

A: AJAX (Asynchronous JavaScript and XML) allows a web page to make asynchronous requests to a server to fetch or update data without requiring a full page refresh. It improves the user experience by enabling dynamic page updates.

Backend Development

These questions focus on the server-side logic and application architecture.

-

Q10: Explain the Node.js event loop and why it's important for asynchronous operations.

-

A: The Node.js event loop is a single-threaded, non-blocking I/O model that handles asynchronous operations. It allows Node.js to perform resource-intensive tasks, like database queries or file system operations, in the background without blocking the main thread. When an asynchronous operation is complete, its callback is placed in the event queue and executed when the main thread is free.

-

Q11: What is middleware in Node.js?

-

A: Middleware is a function that has access to the request object (req), the response object (res), and the next middleware function in the application's request-response cycle. It can execute code, modify the request and response objects, and end the cycle or pass control to the next middleware.

-

Q12: What is the difference between server-side rendering (SSR) and client-side rendering (CSR)?

-

A: SSR generates the HTML on the server and sends a fully-formed page to the client. This is beneficial for SEO and initial page load time. CSR sends a minimal HTML file to the client, and JavaScript is responsible for rendering the content in the browser.

-

Q13: What is RESTful routing and its principles?

-

A: REST (Representational State Transfer) is an architectural style for designing networked applications. RESTful routing uses HTTP methods (GET, POST, PUT, DELETE) to perform CRUD operations on resources, which are identified by a URL. A key principle is that it's stateless, meaning each request from a client contains all the information needed to process it.

-

Q14: Explain MVC (Model-View-Controller) architecture.

-

A: MVC is a design pattern that separates an application into three interconnected components:

-

Model: Manages the application's data, logic, and rules.

-

View: The user interface, responsible for presenting the data to the user.

-

Controller: Handles user input, interacts with the Model, and selects the View to display.

-

Q15: How do you handle file uploads in a Node.js application?

-

A: Using the File System Module is a common way to handle files, but for complex uploads, it's better to use a library like multer, which handles multipart/form-data.

Databases

These questions test your understanding of database concepts and their integration.

-

Q16: Explain the difference between SQL and NoSQL databases. When would you use one over the other?

-

A: SQL databases (like MySQL) are relational, use a structured schema with tables, and are best for structured data and complex queries. NoSQL databases (like MongoDB) are non-relational, have a flexible schema, and are ideal for unstructured data, high-traffic loads, and scalability.

-

Q17: What are CRUD operations?

-

A: CRUD is an acronym for the four basic operations for a database: Create, Read, Update, and Delete. These correspond to HTTP methods: POST for Create, GET for Read, PUT/PATCH for Update, and DELETE for Delete.

-

Q18: What is the purpose of database indexing?

-

A: Database indexing is a data structure technique that improves the speed of data retrieval operations. It's like a book's index, allowing the database to find data without scanning the entire table, thus optimizing queries.

-

Q19: How do you connect a Node.js application to a NoSQL database like MongoDB?

-

A: You use a database driver or an Object-Document Mapper (ODM) like Mongoose. Mongoose provides a schema-based solution to model your application data, enforcing a consistent structure while interacting with MongoDB.

Security & Best Practices

These questions cover important security and development concepts.

-

Q20: How do you handle user authentication and authorization in a full-stack application?

-

A: Authentication verifies the user's identity (e.g., username and password). Authorization determines what the authenticated user is allowed to do. Implement a robust authentication system (e.g., using JWTs) and use middleware to check a user's permissions for specific routes.

-

Q21: What is CORS (Cross-Origin Resource Sharing)? How do you handle it in a Node.js application?

-

A: CORS is a browser security mechanism that restricts a website on one domain from accessing resources on another domain. In Node.js, you can handle CORS by setting appropriate HTTP headers on your server responses, often using a middleware package like cors.

-

Q22: What is the importance of version control? What is Git?

-

A: Version control, like Git, is a system that tracks changes to code over time. It allows multiple developers to collaborate on a project, provides a history of changes, and makes it easy to revert to previous versions if needed. Git is a distributed version control system.

-

Q23: How do you ensure code quality and maintainability?

-

A: Discuss practices like following coding standards, using a linter (e.g., ESLint), performing code reviews, and writing unit tests and integration tests.

-

Q24: What are microservices and how do they differ from a monolithic architecture?

-

A: A monolithic architecture is a single, large application where all components are tightly coupled. A microservices architecture is an approach where a large application is broken down into a suite of small, independent services that communicate with each other. Microservices are easier to scale, deploy, and maintain.

-

Q25: Explain the importance of a CI/CD pipeline.

-

A: A CI/CD (Continuous Integration/Continuous Deployment) pipeline automates the process of building, testing, and deploying an application. This ensures that code changes are consistently integrated and delivered, reducing manual errors and accelerating the release cycle.

